

Software Requirements Management

Document Type: C Language Coding Standards
Scope: All software within the company coded in C language
Effective Date: Effective from the date of release
Version: V1.0

Document Revision History

Version	Change Date	Author	Reviewer	Description
V1.0	2025-11-20	王天楠	杜龙彬	Initial release, includes the basics

Document Overview

This specification defines the basic principles, rules and recommendations for writing C programs. It provides concrete guidance on clarity, simplicity, testability, security, performance and portability of C programming.

Composition of the Specification

- Principles: Guiding ideas that must be followed when programming
- Rules: Mandatory conventions to be followed when programming
- Recommendations: Conventions that should be considered when programming

Main Contents

1. Header file conventions
2. Function design conventions
3. Identifier naming and definitions
 - Variable naming
 - Macro naming
 - Function naming
 - Struct naming
4. Layout and formatting
5. Commenting conventions
6. Testability requirements
7. Interrupt handling conventions

Implementation Requirements

- Newly written code must comply with this specification
 - When modifying existing code, it should be progressively brought into compliance with this specification
 - All developers must be familiar with and follow this specification
-

Prepared by: Software Development Department

Preparation Date: November 2025

Classification: Internal

C Language Coding Standards

Scope

This specification applies to all software within the company developed in C. It takes effect from the date of release. New and modified code should comply with this specification.

Introduction

This specification defines the basic principles, rules and recommendations for writing C programs. It provides concrete guidance on clarity, simplicity, testability, security, performance and portability of C programming.

0 Specification Notes

0.1 Preface

To improve product code quality and guide developers to write concise, maintainable, reliable, testable, efficient and portable code, the coding standards revision working group analyzed and summarized typical coding issues in our company and referred to recent industry standards. We have reorganized and optimized our coding standards and produced this document. There will be a full version and a condensed version; the full version will include more examples and detailed explanations.

0.2 Overall Coding Principles

1. Clarity first

Readability is essential for maintainable and refactorable code. Code is primarily written for people to read; good code should read like prose.

"Programs must be written for people to read, and only incidentally for machines to execute."

— Harold Abelson and Gerald Jay Sussman

"Programming should be human-centered, computers second."

— Steve McConnell

2. Simplicity is beautiful

Simplicity makes code easier to understand and implement. Longer code is harder to read and more likely to introduce bugs during changes.

3. Choose an appropriate style and remain consistent with existing code

The benefits of a shared coding style across the product outweigh the cost of enforcing it.

0.3 Implementation and Interpretation

This specification applies to all software developed in C within the company and is effective from the date of release. New and modified code should follow this specification.

0.4 Terminology

- Principle: Guiding ideas to be followed when programming
 - Rule: Mandatory conventions to be followed when programming
 - Recommendation: Conventions that should be considered when programming
 - Explanation: Necessary explanation of a principle/rule/recommendation
 - Example: Positive and negative examples illustrating the guidance
-

1 Header Files

Header file design reflects much of the system design. Poor header layout is a major cause of long compilation times.

Definition

- Dependency: If x.h includes y.h, x depends on y. Dependencies are transitive (x.h includes y.h and y.h includes z.h means x depends on z through y). Dependencies increase compilation time. While some dependency is unavoidable, poor design can make the dependency graph very complex so that any change causes recompilation of large parts of the system.

Principles

1.1 Headers should contain declarations of interfaces, not implementations

Explanation:

- Headers are the external interfaces of a module; they should contain declarations intended for external use.
- Variable definitions should not be placed in headers; they belong in .c files. Avoid exposing implementation details via global variables declared in headers. If globals are necessary, define them in a .c file and only declare them in a header when absolutely required.

1.2 Headers should have single responsibility

Explanation: Large or overly complex headers and complex dependencies are a primary cause of long compile times. Avoid overly large headers with multiple responsibilities and circular dependencies.

1.3 Headers should include in a stable direction

Explanation: Include relationships are dependencies. Prefer letting unstable modules depend on stable modules so that changes in unstable modules don't affect (compile) stable modules.

Rules

1.1 Each .c file should have a same-named .h file

Explanation: Use the .h file to declare interfaces that need to be publicly exposed.

1.2 Forbid circular dependencies between headers

Explanation: Circular includes (e.g., a.h includes b.h, b.h includes c.h, c.h includes a.h) can cause changes in any header to trigger recompilation of all dependent code. One-way dependencies are acceptable.

1.4 Headers must be self-contained

Explanation: Any header can be compiled independently.

1.5 Always add include guards in headers

Explanation: Use #define guards to prevent multiple inclusion. Naming format: FILENAME_H, recommended PROJECTNAME_PATH_FILENAME_H.

1.6 Do not define variables in headers

1.7 Use interfaces provided by other .c files only by including their header files

Explanation: Forbid using extern to access external function interfaces or variables directly in .c files instead of including headers.

2 Functions

Principles

2.1 A function should do only one thing

Explanation: Functions that implement multiple unrelated responsibilities are harder to develop, use and maintain. Keep function responsibilities focused to improve readability, testability and modifiability.

2.2 Duplicate code should be refactored into functions

Explanation: Refactoring repeated code into functions reduces maintenance cost.

Rules

2.1 Avoid excessively long functions; new functions should not exceed 50 lines (non-empty, non-comment)

Exception: Some algorithm implementations may exceed 50 lines.

2.2 Avoid deep nesting; new functions should not nest more than 4 levels

2.3 Reentrant functions should avoid shared variables; if needed, protect them with mutexes

2.4 Remove dead code (unused functions and variables) promptly

Explanation: Dead code consumes space and may affect functionality and performance; it complicates testing and maintenance.

2.5 Functions declared and defined at file scope should be marked static unless externally visible

Explanation: If a function is called only within the same file, declare it static. static confines visibility to the declaring file and avoids symbol conflicts.

2.6 Limit function parameters to no more than 5

Explanation: Excessive parameters increase coupling, make maintenance and testing harder. If more than 5 are needed, consider splitting functionality into multiple functions.

3 Identifier Naming and Definitions

Principles

3.1 Names should be clear, meaningful, use complete words or widely understood abbreviations, and avoid ambiguity.

Explanation: Prefer descriptive names over saving space; clarity aids understanding.

Example:

```
// Good names
int error_number;
int number_of_completed_connection;

// Bad names
int n;
int nerr;
int n_comp_conns;
```

3.2 Use correct antonym pairs for mutually exclusive variables or opposite actions, e.g.:

```
add/remove  up/down  create/destroy
min/max     next/previous  send/receive
```

3.3 Avoid numerical suffixes in names unless logically required. Example of confusing names:

```
#define EXAMPLE_0_TEST
#define EXAMPLE_1_TEST
```

Prefer meaningful names:

```
#define EXAMPLE_UNIT_TEST
#define EXAMPLE_ASSERT_TEST
```

-
- 3.4 Keep naming style consistent throughout.
 - 3.5 If special conventions or abbreviations are used in names, document them.
 - 3.6 Avoid project-specific prefixes in names to promote reuse.

4 File Naming Rules

Principles

- 4.1 Files and directories should be organized clearly to avoid conflicts.
- 4.2 File and directory names should reflect module functionality, reduce cross-module dependencies, and facilitate team maintenance.

Rules

- 4.1 Source (.c) and header (.h) filenames use lowercase with underscores; directories are organized by functionality (e.g., drivers, system).

Example layout:

```
project/
├── drivers/           // Hardware driver modules (UART, SPI, GPIO, etc.)
│   ├── uart_driver.c
│   └── spi_driver.h
├── system/           // System core modules (task scheduler, watchdog, etc.)
│   ├── task_scheduler.c
│   └── watchdog.h
├── application/      // Application modules (sensor processing, display control,
etc.)
│   └── sensor_process.c
└── main.c            // Main entry point
```

5 Variable Definitions

Principles

- 5.1 Variable naming: use lower camelCase (scope-distinguishing) with a lowercase initial letter and capitalized subsequent words; use prefixes/modifiers to indicate scope (local / static local).

Variable naming format: **<scope>_<type>name**

Scope prefix table

Scope	Prefix	Example
Struct member	m_	m_redComponent
Static variable	s_	s_instanceCount
Global variable	g_	g_systemTick
Local variable	(none)	tempValue

Type prefix table

Type	Prefix	Example
Pointer	p	pBuffer
Enum	e	eDeviceState
Struct	t	tSensorData
Boolean	b	blsReady
Float	f	fTemperature
Double	d	dPrecisionValue
Char	c	cKeyInput
Unsigned integer	u	uDataLength
Signed integer	i	iCounter
Short	s	sSmallValue
Long	l	lLongValue

5.2 Use i, j, k for local loop counters; len, num for local length/count variables; temp, tmp for temporary intermediates

Example:

```
// Single loop
for (uint8_t i = 0; i < 10; i++)
{
    printf("%d ", i);
}
```

Correct examples:

```
// Correct: local variable representing a sensor value
uint32_t sensorValue;
```

```
// Correct: static local variable representing packet counter
static uint8_t packetCounter;
```

Incorrect examples:

```
// Incorrect: mixed snake_case and PascalCase
int Sensor_Value;

// Incorrect: local variable using PascalCase
float Temperature;
```

5.3 Global variables: use g_ prefix.

Explanation: All global variables (including static globals) should have g_ prefix to avoid cross-file naming conflicts; hardware-related variables should be volatile to prevent compiler optimizations.

Correct examples:

```
// Correct: global variable, system tick counter (volatile for hardware variables)
volatile uint32_t g_systemTickCount;

// Correct: static global variable, packet error counter
static uint16_t g_packetErrorCount;
```

Incorrect example:

```
// Incorrect: missing g_ prefix, hard to identify as global
uint32_t SystemTick;
```

6 Macro Naming Rules

Principles

6.1 For numeric or string constants, prefer ALL_CAPS with underscores; enums are also recommended to follow this style.

Example:

```
#define PI_ROUNDED 3.14
```

7 Function Naming: PascalCase (Verb first)

7.1 Start with a capital letter, follow PascalCase; start with a verb (e.g., Init, Read, Configure) to indicate action.

Correct example:

```
// Correct: clearly initializes UART peripheral, follows PascalCase and verb-first
void InitUartPeripheral(void);
```

Incorrect examples:

```
// Incorrect: snake_case not compliant
void init_uart(void);

// Incorrect: inconsistent capitalization
void ReadtemperatureSensor(void);
```

8 Custom Type Naming Conventions

Rules

- Struct types: use prefix t + PascalCase to identify struct types
- Enum types: use prefix e + PascalCase to identify enum types
- Type names: use clear prefixes to indicate type category for readability and maintainability

Example:

```
typedef enum {
    eSENSOR_OFF,
    eSENSOR_STANDBY,
    eSENSOR_ACTIVE
} eSensorState;

typedef struct {
    int nId;
    char szName[50];
    float fCurrentValue;
    eSensorState eState;
} tSensor;

// Global sensor list
tSensor g_arrSensors[MAX_SENSORS];
```

9 Layout and Formatting

Rules

9.1 Use indentation for code blocks; each level is 4 spaces.

Explanation: Configure editors/IDEs to convert TAB to spaces and show tabs to avoid mistakes.

9.2 Add blank lines between relatively independent blocks and after variable declarations.

9.3 Avoid overly long statements; if unbreakable, split lines. The line length policy can be decided by the product team.

Guidelines for line breaks:

- When breaking lines, increase indentation by one level for readability
- Break at low-precedence operators when possible
- Place the operator at the start of the new line
- Prefer putting a complete logical statement on one line where possible

Example:

```
#include <stdio.h>

int main()
{
    int a = 10;
    int b = 20;

    int result = a + b + 100 + 200 + 300 + 400 +
                500 + 600 + 700 + 800 + 900 + 1000;

    printf("The result is %d", result);

    return 0;
}
```

```
// Break long expressions at low-precedence operators
int complex_result = (a * b) + (c * 2) +
                    (a + b + c) * 3 -
                    (a - b) / 2;
```

10 Interrupt Handler Conventions

Principles

Interrupt handlers should follow minimalism and standardized naming to ensure timely and reliable responses.

Rules

Names must strictly follow the chip vendor's conventions to ensure correct interrupt vector mapping.

Examples:

- USART1_IRQHandler — USART1 interrupt service routine
- TIM2_IRQHandler — Timer 2 interrupt service routine
- EXTI0_IRQHandler — External interrupt 0 service routine

Code conventions

Rule 1: Standardized function names

```
// Correct: follows vendor naming conventions
void USART1_IRQHandler(void);
void TIM2_IRQHandler(void);
void EXTI0_IRQHandler(void);

// Incorrect: custom names will prevent interrupts from being handled
void MyUartInterrupt(void);
void Timer_ISR(void);
```

11 Commenting Conventions

Principles

Explanation: Good comments help reader understanding, explain what code does, its purpose and approach. Aim for meaningful, concise and accurate comments. Add comments while coding and update them when code changes. Remove obsolete comments. Use `"/"` in both assembly and C; avoid block comments `"/* */"` except during debugging.

11.1 File header comments: include core information

Explanation: Each `.c` and `.h` file should have a file header comment containing filename, project, copyright, purpose, author and date.

```
/*
*****
 * @copyright: © 2025, XX Co., Ltd. All rights reserved.
 * @file: websocket.c
 * @description: Implements 485 protocol parsing, frame packaging and sending
logic
 * @author: XXX
 * @date: 2025-08-14
 * @version: V1.0
*****
*/
```

11.2 Function comments: required for complex functions

Explanation: Complex functions (algorithms, hardware configuration) should have comments explaining core logic; simple functions (like simple assignments) may omit comments.

```
/**
 * @brief Calculate CRC16-CCITT checksum (for serial data validation)
 * @note Polynomial: 0x1021 (x^16 + x^12 + x^5 + 1), init: 0xFFFF, no reflection
 * @param pData: pointer to the data to be checked (input, non-NULL)
 * @param length: length of data to check (input, >0)
 * @return uint16_t: computed CRC value
 */
uint16_t CalculateCrc(const uint8_t* pData, uint16_t length) {
    // ... implementation ...
}
```

11.3 Code comments

Commenting conventions

Rules

Code comments should be adjacent to the code they document, placed above it or on the right; do not place comments below the code.

Explanation

- If placed above, separate from the preceding code with a blank line
- Small comments may appear at the end of the line
- Multiple comments in a function should be left-aligned
- Longer comments should be placed above and left-aligned with the code they document

Comment position examples

Above comments (for longer explanations)

```
// Separated by a blank line
/**
 * @brief Initialize SPI peripheral
 * @note Configure as master mode, clock polarity = 0, clock phase = 0
 */
void InitSPI(void)
{
    SPI_InitTypeDef spi_init;
    spi_init.Mode = SPI_MODE_MASTER;
    spi_init.CLKPolarity = SPI_POLARITY_LOW;
    spi_init.CLKPhase = SPI_PHASE_1EDGE;
```

```
// ... other configuration ...  
}
```

Multi-line comments left-aligned

```
void HandleNetworkPacket(void)  
{  
    ParseHeader(packet);    // Parse packet header  
    ValidateChecksum(packet); // Validate checksum  
    ProcessPayload(packet);  // Process payload  
    SendAck(sender_id);      // Send acknowledgment  
}
```

Branch comment rules

```
// Iterate over sensor array, read and process data  
for (int i = 0; i < SENSOR_COUNT; i++)  
{  
    // Check if the sensor is online  
    if (sensors[i].status == ONLINE)  
    {  
        // Read raw sensor data  
        raw_data = ReadSensor(i);  
  
        // Apply filtering algorithm  
        filtered_data = ApplyFilter(raw_data);  
  
        // Update display data  
        UpdateDisplay(i, filtered_data);  
    }  
}
```

Software Testability Design Principles

Principle 12.1: Module testability design

Core requirement: Modules should be clearly separated, interfaces explicit, low coupling, with clear inputs and outputs; otherwise unit testing is difficult.

Preconditions for Unit Test Implementation

1. Interface definition specification

- Clear definitions: interface parameters and return types are explicit
- Complete definitions: cover all necessary inputs and outputs

- Stable definitions: low change frequency and backward compatible

2. Clear acceptance criteria

- Preconditions: required environment state before execution
- Input data: explicit input ranges and formats
- Expected results: verifiable output standards

3. Internal state observability

- State queryable: critical internal state can be obtained
- Data modifiable: key data can be set for tests
- Process monitorable: execution can be traced

4. Execution path control

- Single entry: atomic functions have a single clear entry point
- Single exit: atomic functions have a determined exit point
- Clear path: execution flow is predictable

5. Dependency management

- Centralized dependencies: external dependencies are handled centrally
 - Minimize global variables:
 - Reduce use of globals
 - Or encapsulate them (singleton, service locator)
 - Provide controllable dependency injection mechanisms
-